

WEEK8

DRIVECARE PRO

Automotive Mobile App for Vehicle Owner Self-Service

HTML + CSS + JavaScript + Node.js + Express.js + MongoDB

Aim:

DriveCare PRO is a digital vehicle self-service platform for customers, service advisors and administrators. It supports registration and login, vehicle management, service appointments, digital service records, service centers, roadside assistance, notifications and advisor/workshop functions. The system uses HTML, CSS and JavaScript on the frontend, Node.js and Express REST APIs on the backend, and MongoDB with Mongoose for data management.

Solution:

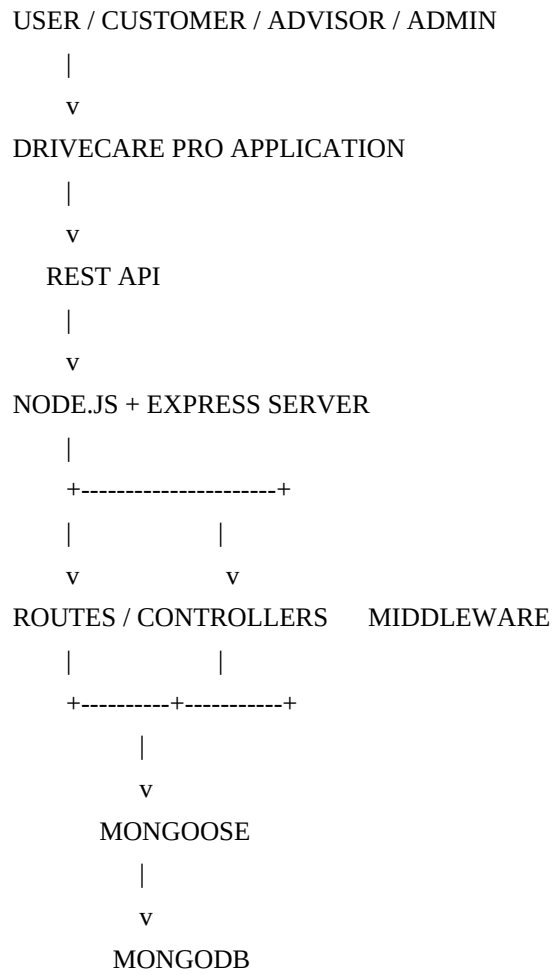
1. Project Objective

DriveCare PRO digitizes vehicle and service information, provides online booking and status tracking, and connects users with service centers and roadside assistance.

2. Technology Stack

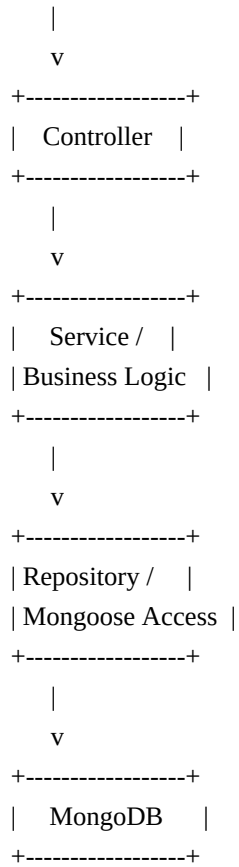
Technology	Purpose
HTML / CSS / JavaScript	Frontend
Node.js / Express.js	Backend and REST APIs
Routes / Controllers / Middleware	Business layer
Mongoose / MongoDB	Database
JWT	Authentication and protected routes
Postman	API testing
GitHub	Version control and project tools

3. Architecture Dashboard



4. Internal Layered Architecture

REST Request



Business processing uses routes, controllers and middleware between the REST API and database layer.

5. Project Structure

drivecare-pro/

- frontend/

- ■■■ HTML

- ■■■ CSS

- ■■■ JavaScript

- backend/

- ■■■ routes/

- ■■■ controllers/

- ■■■ middleware/

- ■■■ server/

- models/

- ■■■ User

- ■■■ Vehicle

- ■■■ Appointment

- ■■■ ServiceRecord

- ■■■ Notification

- ■■■ ServiceCenter

- ■■■ RoadsideRequest

- tools/

- Postman

- GitHub

6. Database Model

USER • VEHICLE • APPOINTMENT • SERVICERECORD • NOTIFICATION • SERVICECENTER • ROADSIDE REQUEST

7. Database Setup

MongoDB is used with Mongoose for persistent application data.

8. Backend Configuration

Backend: Node.js + Express.js

API style: REST APIs

Business layer: Routes + Controllers + Middleware

Database: MongoDB + Mongoose

Security: JWT authentication

Testing: Postman

Version control: GitHub

The supplied project documents a centralized backend flow connecting the user, REST API, Node.js/Express server and MongoDB database.

9. Main Program / Backend

The DriveCare PRO backend is implemented using Node.js and Express.js.

Main backend responsibilities:

- Receive REST API requests.
- Route requests to the correct controller.
- Apply middleware and protected-route checks.
- Perform vehicle, appointment and service operations.
- Use Mongoose to access MongoDB.
- Return REST responses to the client.

The supplied project identifies Routes, Controllers and Middleware as the business layer.

10. Authentication & User Management

Registration with user details



Login using email + password



Password verification



JWT token generation



Authorization header



Protected API routes

Roles documented in the project:

Customer • Advisor • Admin

Only authenticated users can access protected application functions.

11. Vehicle & Appointment Management

Vehicle and appointment flow:

1. Select vehicle.
2. Select service center.
3. Choose service type.
4. Select date and time slot.
5. Add pickup / drop requirements.
6. Add estimated cost and customer notes.
7. Track appointment status.

STATUS FLOW:

Pending → Confirmed → In-Progress → Completed

12. API & Controller Functions

The REST API provides the application interface for vehicle-service operations.

Documented functions include:

- Registration and login.
- Vehicle management.
- Service appointment booking.
- Digital service records.
- Service-center information.
- Notifications.
- Roadside assistance requests.

Controllers process API requests while routes direct requests and middleware supports authentication and request protection.

13. Notifications & Emergency Services

DriveCare PRO supports:

- Appointment notifications.
- Roadside assistance requests.
- Service-center information.
- Protected application functions.

Roadside assistance provides a digital channel for emergency requests, while notifications help customers receive timely service and appointment information.

14. Testing & Expected Results

Testing documented for DriveCare PRO:

Test	Expected Result

CRUD operations	Data operations work correctly
Input validation	Invalid input is rejected
Protected API routes	Only authenticated access
REST/database integration	API connects with database

Key outcomes:

- Authentication and protected routes
- Vehicle management
- Appointment booking and status tracking
- Digital service records
- Service centers and notifications
- Roadside assistance

15. Security & Data Flow

LOGIN



JWT TOKEN



AUTHORIZATION HEADER



MIDDLEWARE



CONTROLLER



DATABASE



PROTECTED RESPONSE

Security features documented in the project:

JWT authentication, authorization headers, protected routes and password verification.

16. Results & Conclusion

Results:

DriveCare PRO provides a centralized digital experience connecting vehicle owners, service centers and backend services. The system covers authentication, vehicle management, appointment booking, service records, service centers, notifications and roadside assistance.

Conclusion:

The platform brings traditional vehicle-service tasks into one digital system, reducing dependence on manual booking and scattered records while providing online appointment tracking and a digital emergency channel.

Future Scope

- AI-based maintenance prediction
- Live roadside assistance tracking
- Online payment integration
- Mobile application using Flutter
- Advanced analytics

26. Out-of-Stock Test

If availableStock is zero, the service must not create an order.

```
POST http://localhost:8080/orders
```

```
{
  "userId": 502,
  "bookId": 103
}
```

Expected HTTP response: 409 Conflict

```
{
  "status": 409,
  "error": "OUT_OF_STOCK",
  "message": "Book 103 is out of stock"
}
```

27. Validation Test

Stock values must not be negative. @Min(0) rejects values below zero.

```
@Min(value = 0, message = "Available stock cannot be negative")
private Integer availableStock;
```

28. End-to-End Gateway Dashboard

```
CLIENT
|
| POST /gateway/orders
| { userId: 501, bookId: 101 }
v
FASTAPI GATEWAY
|
| HTTP POST /orders
v
SPRING BOOT
|
+--> Find Inventory
|
+--> availableStock > 0 ?
|
|      YES
|      |
|      +--> availableStock - 1
|      |
|      +--> Create BORROWED Order
|
v
POSTGRESQL
|
v
JSON RESPONSE
{ id: 1, userId: 501, bookId: 101, status: "BORROWED" }
```

29. FastAPI Gateway Example

```
from fastapi import FastAPI
import httpx

app = FastAPI()

SPRING_SERVICE = "http://localhost:8080"

@app.post("/gateway/orders")
async def create_order(order: dict):

    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{SPRING_SERVICE}/orders",
            json=order
        )

    return response.json()
```

30. Complete Testing Checklist

Test	Expected Result	Status
Spring Boot starts	Port 8080 active	PASS
PostgreSQL connection	Application connects	PASS
GET inventory	Returns stock JSON	PASS
POST order	Creates BORROWED order	PASS
Stock decrement	Available stock decreases by 1	PASS
Zero stock	HTTP 409	PASS
Negative stock validation	Rejected	PASS
Database persistence	Rows remain in PostgreSQL	PASS
Gateway forwarding	FastAPI reaches Spring Boot	PASS

31. Common Problems and Solutions

Problem	Likely Cause	Solution
Connection refused	PostgreSQL not running	Start PostgreSQL service
Password authentication failed	Wrong DB password	Update application.yml
Port 8080 in use	Another service uses 8080	Stop it or change server.port
Book not found	No inventory row	Insert the book into inventory
Out of stock	availableStock = 0	Add stock or choose another book
Maven command not found	Maven not installed/PATH issue	Install Maven or use Maven wrapper

32. Learning Outcomes

- Controller–Service–Repository architecture
- Spring dependency injection and IoC
- REST API development
- Spring Data JPA and automatic CRUD
- PostgreSQL database integration
- Validation with @Min and @NotNull
- Global exception handling
- Transactional business logic using @Transactional
- Postman API testing
- FastAPI-to-Spring Boot microservice communication
- Polyglot architecture using Python, JavaScript and Java

33. Final Command Sheet

```
# Verify Java
java -version
```

```
# Verify Maven
mvn -version
```

```
# Build
mvn clean install
```

```
# Run
mvn spring-boot:run
```

```
# API
GET http://localhost:8080/inventory/101
```

```
POST http://localhost:8080/orders
```

```
# PostgreSQL  
CREATE DATABASE bookflow;
```

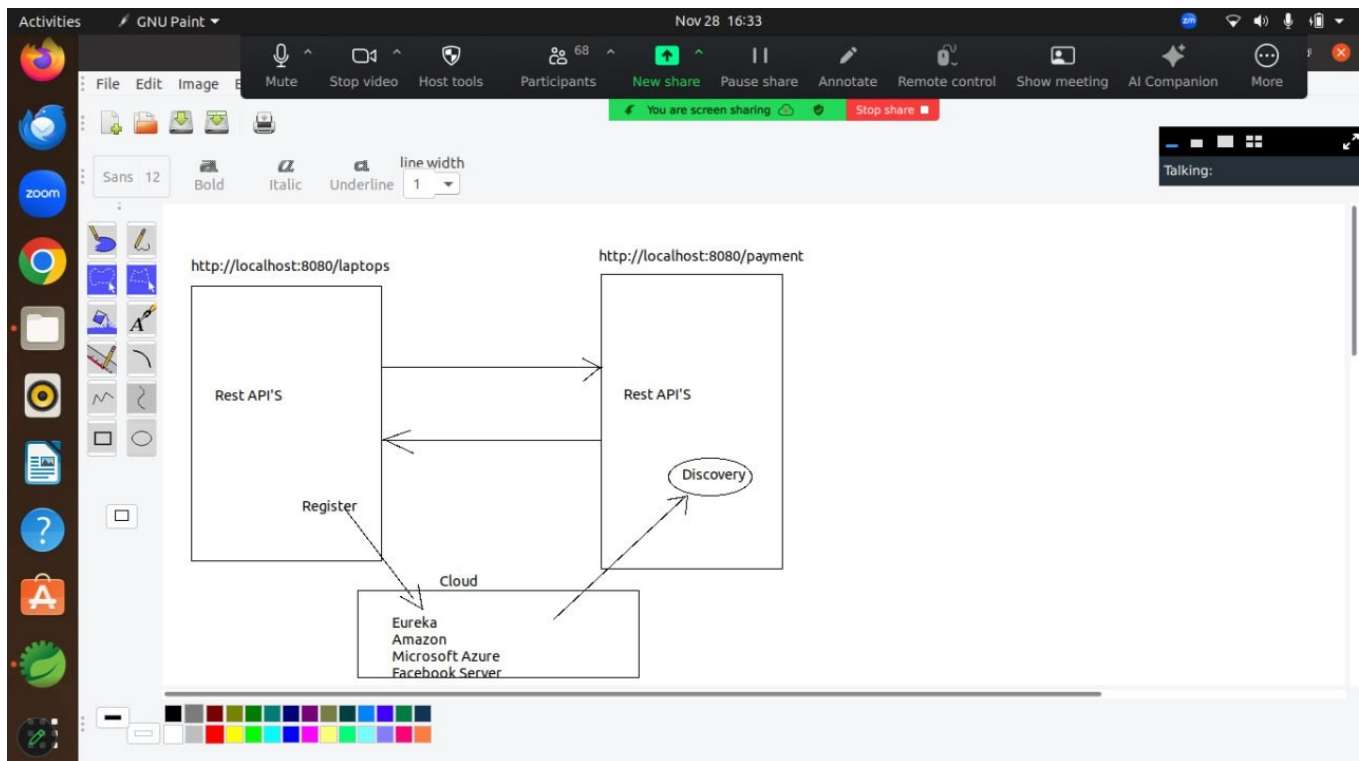
```
# Verify data  
SELECT * FROM inventory;  
SELECT * FROM orders;
```

One more example for practice:

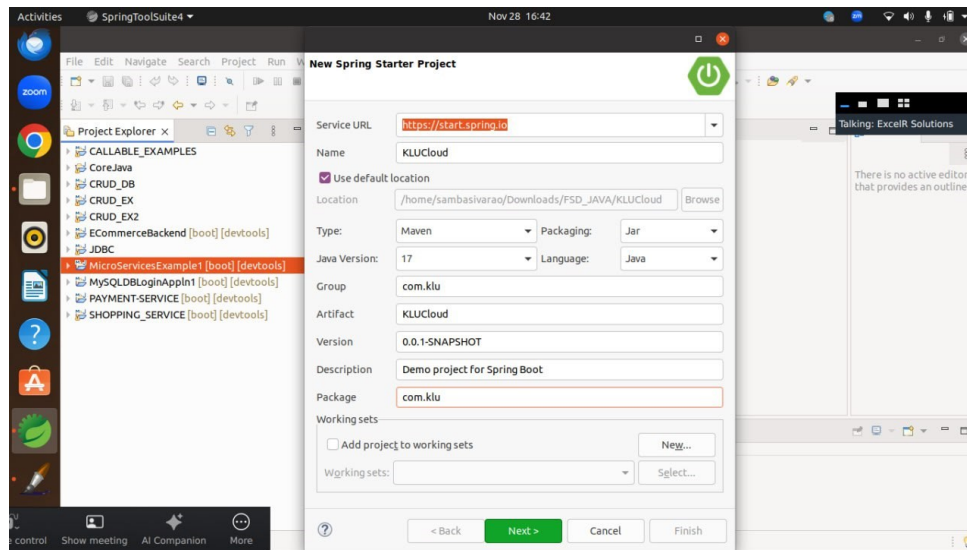
Microservices using spring boot

Creation of Eureka Server

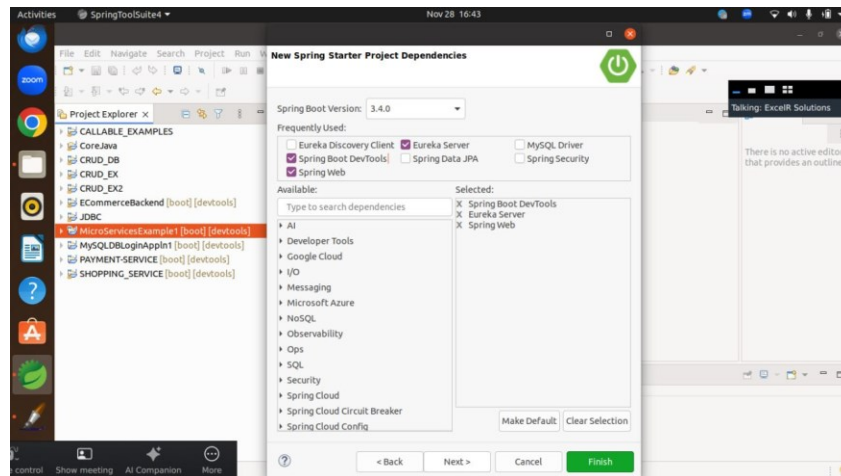
Step-1



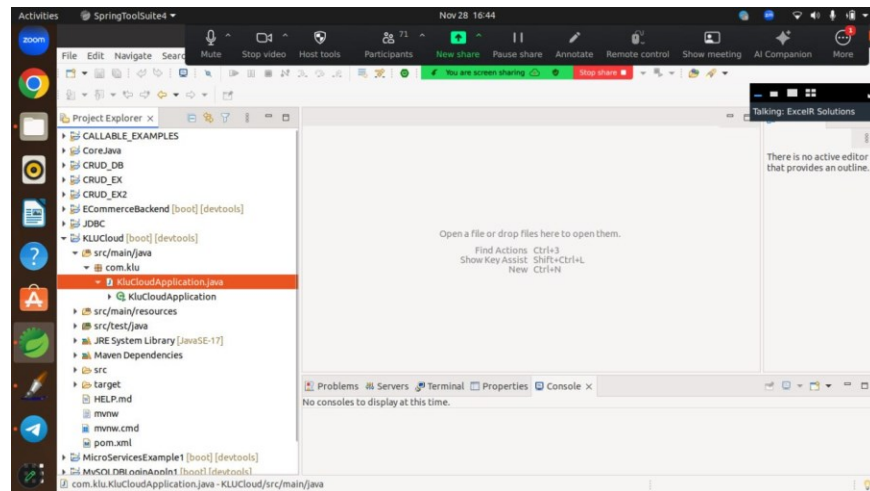
Step-2



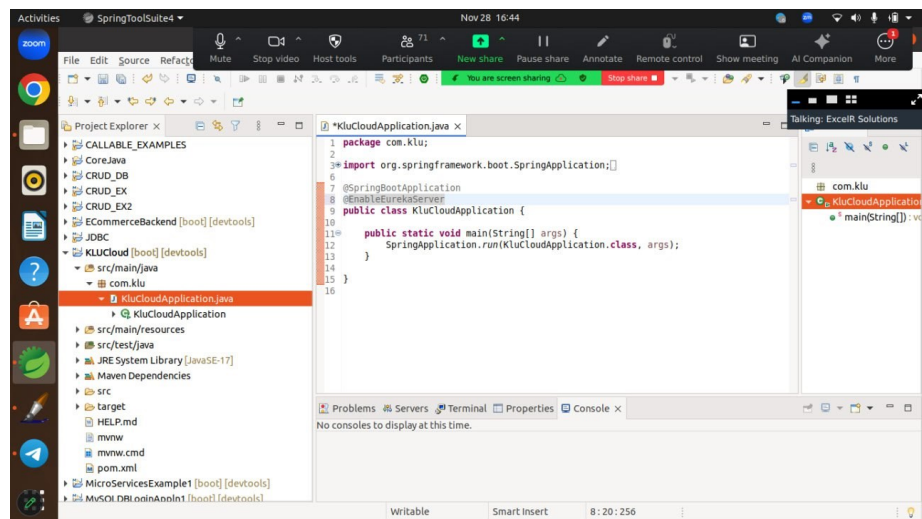
Step-3



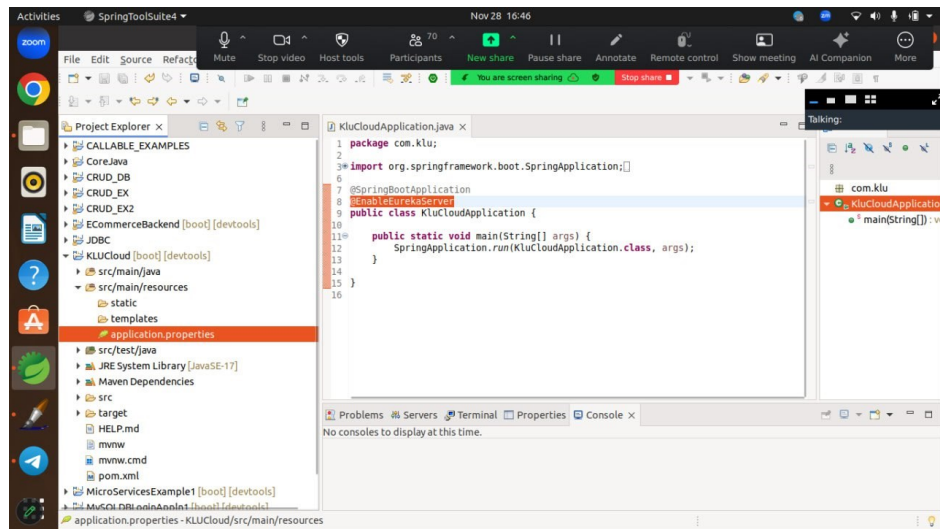
Step-4



Step-5



Step-6



Step-7

Change application.properties file to application.yml

Application.yml

eureka:

client:

register-with-eureka: false

fetch-registry: false

server:

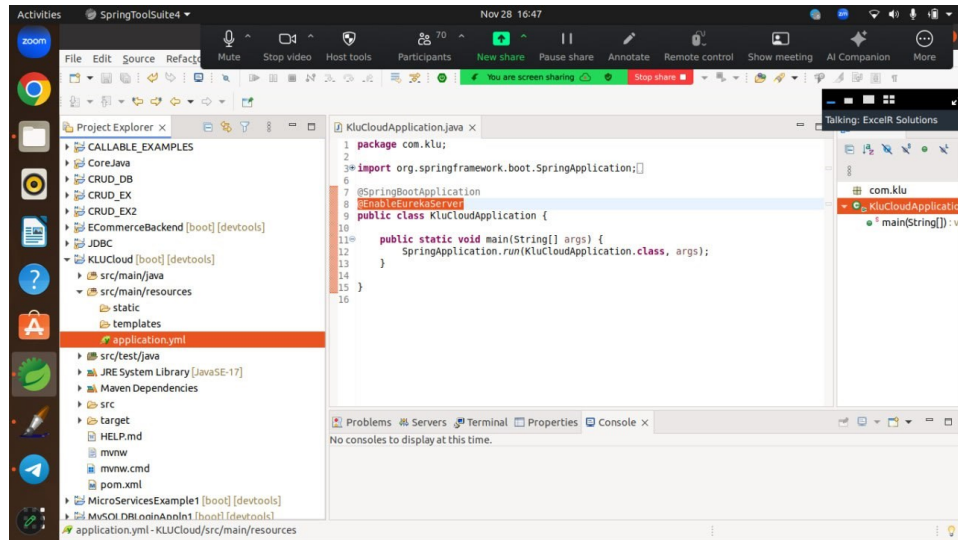
port: 8761

spring:

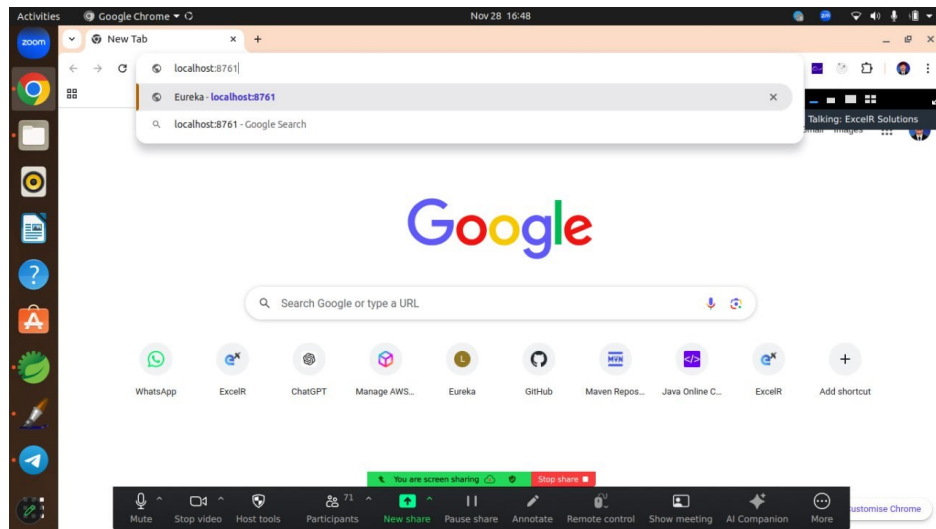
application:

name: EUREKA-CLOUD-SERVER

Step-8



Step-9



Step-10

Activities

Google Chrome

Nov 28 16:49

zoom

Eureka

localhost:8761

spring Eureka

HOME LAST 1000 SJI Talking: ExcelR Solutions

System Status

Environment	test	Current time	2024-11-28T16:48:48 +0530
Data center	default	Uptime	00:00
		Lease expiration enabled	false
		Renews threshold	1
		Renews (last min)	0

DS Replicas

localhost

Insta

You are screen sharing

Stop share

Mute

Stop video

Host tools

Participants

New share

Pause share

Annotate

Remote control

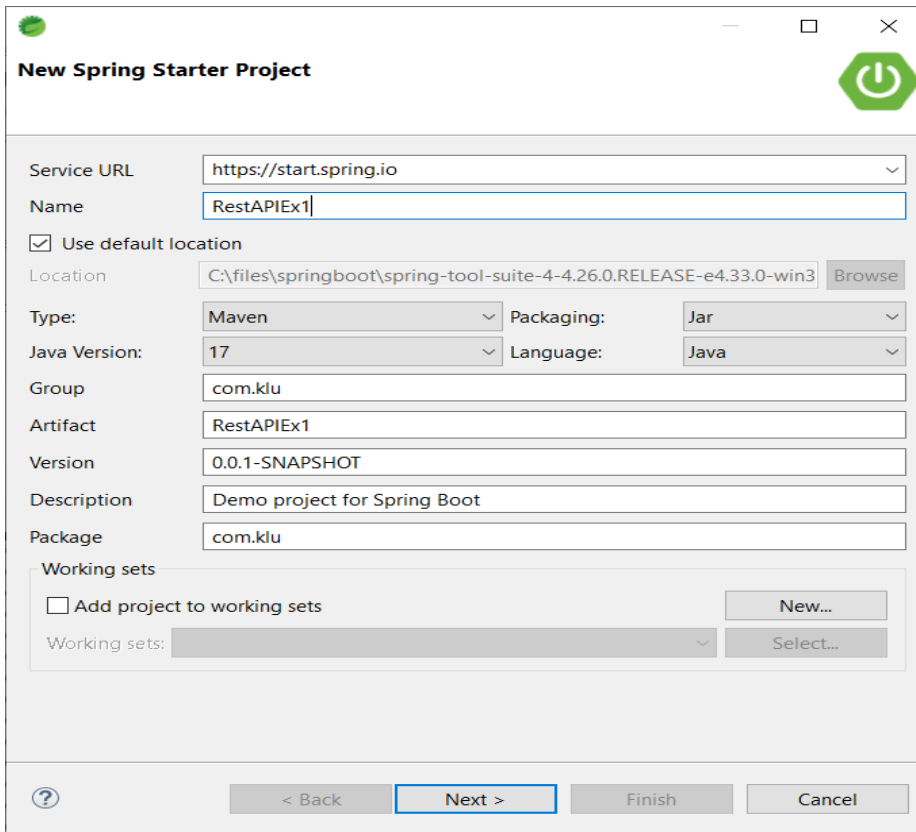
Show meeting

AI Companion

More

Creation RESTApi project and run in Eureka Server

Step-1



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

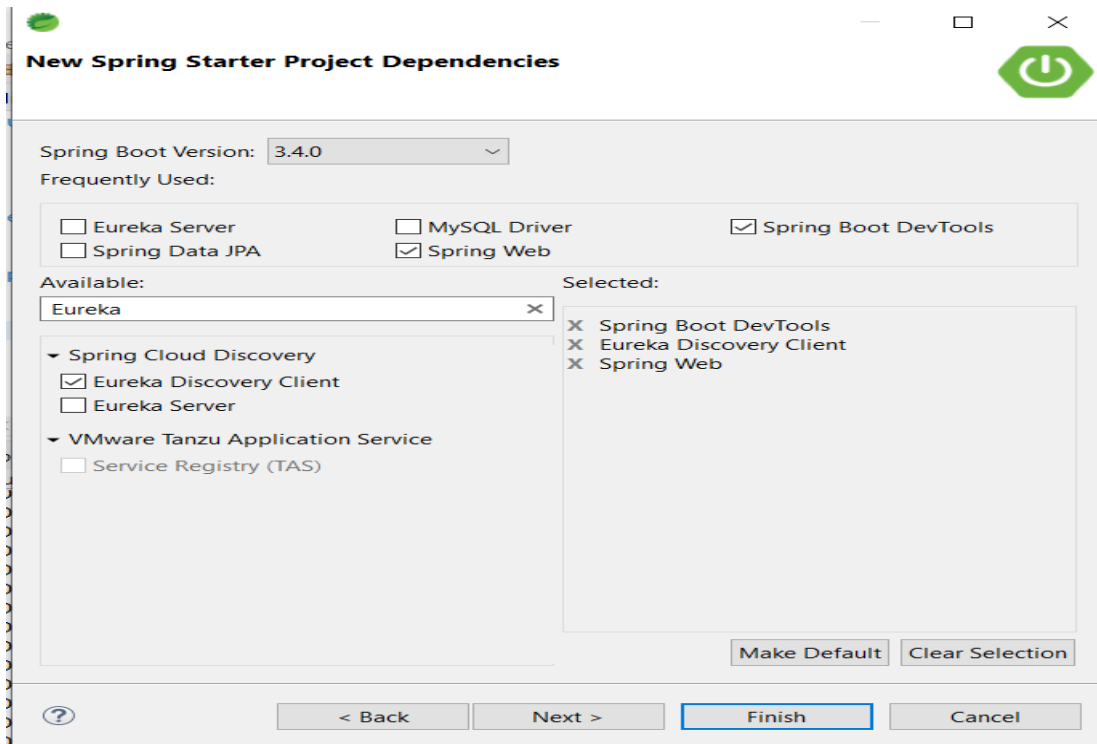
Package:

Working sets

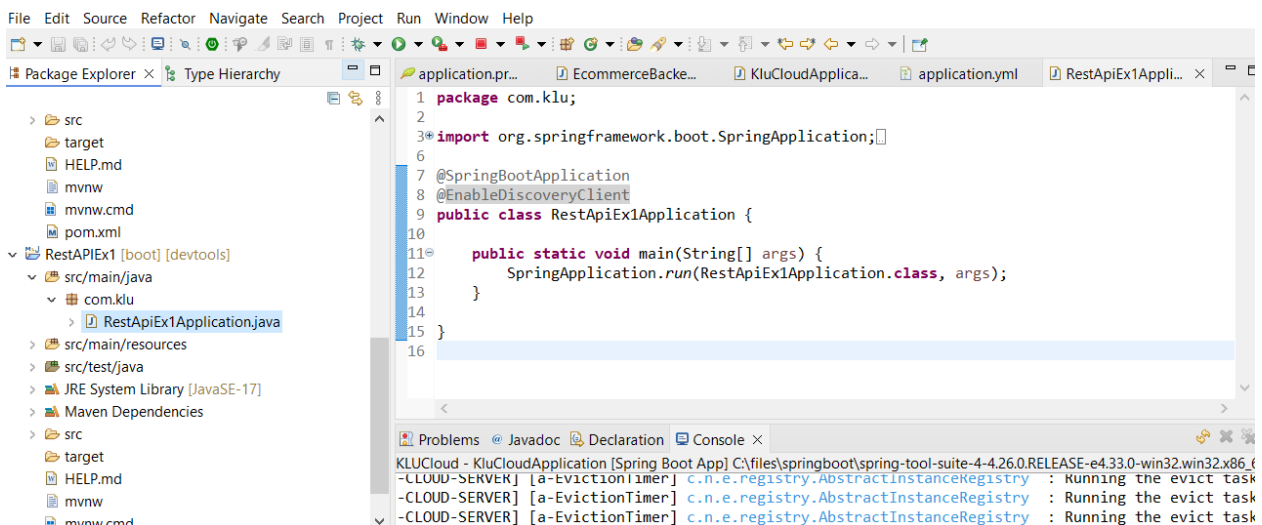
☐ Add project to working sets

Working sets:

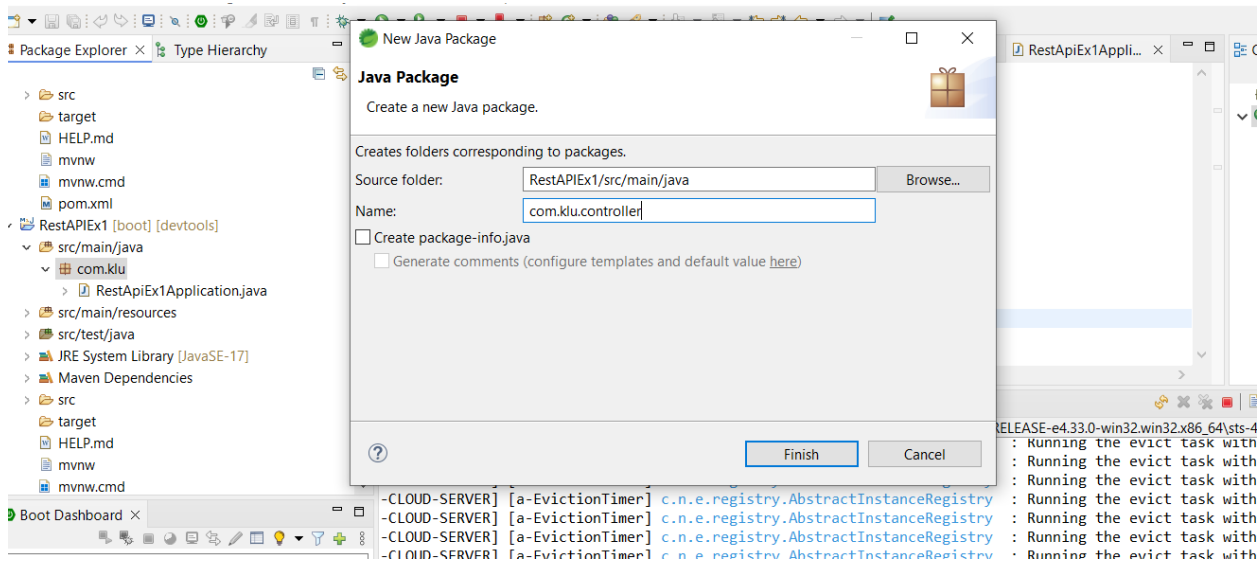
Step-2



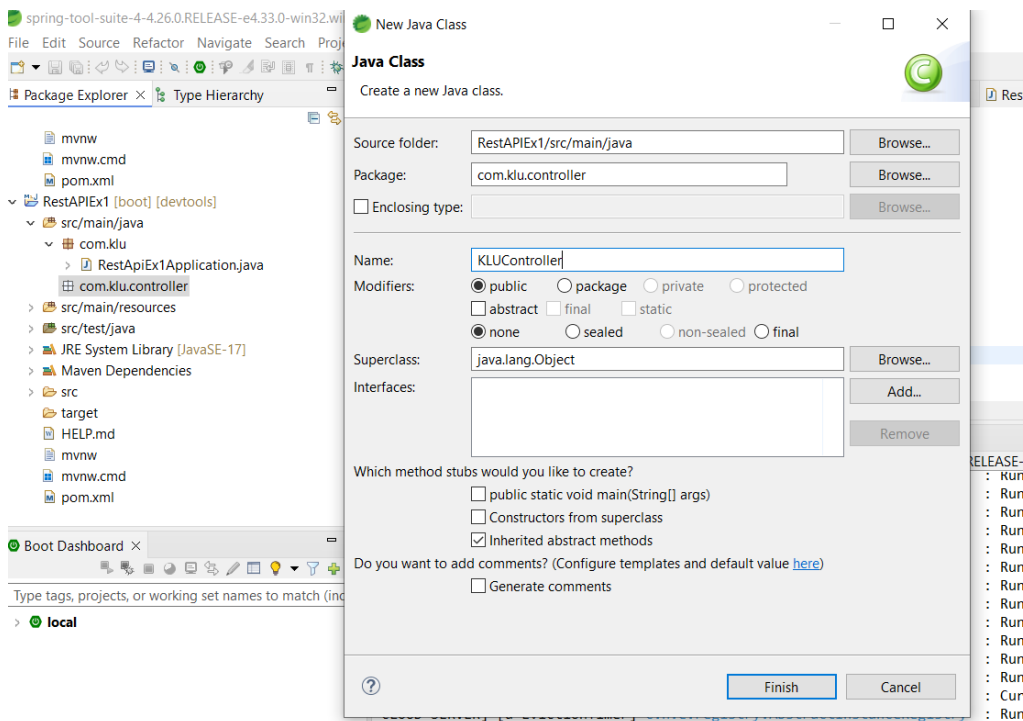
Step-3



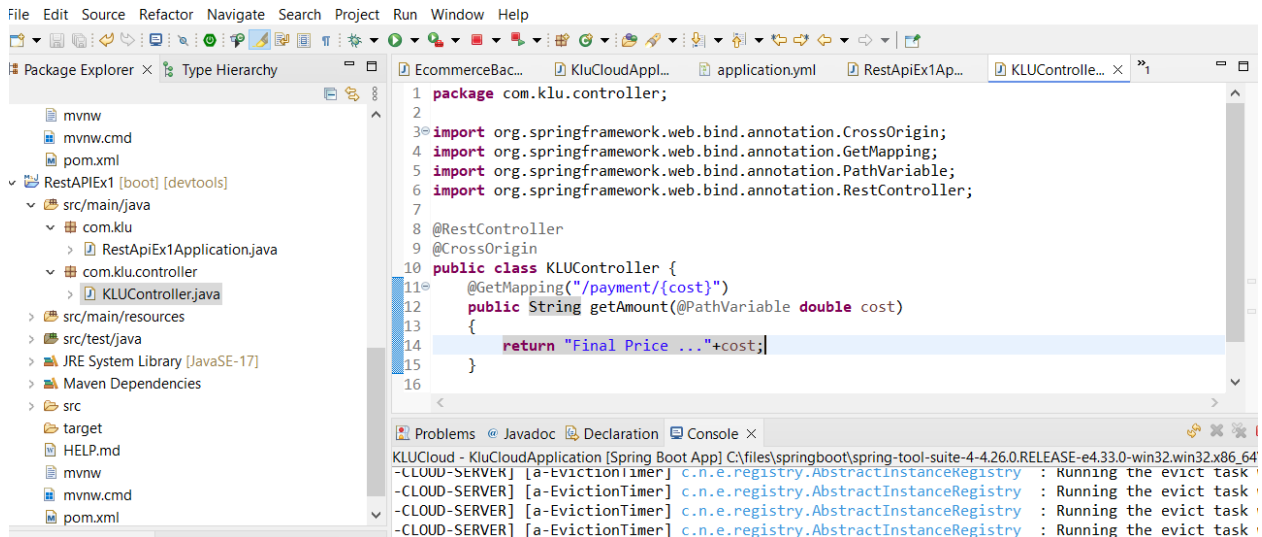
Step-4



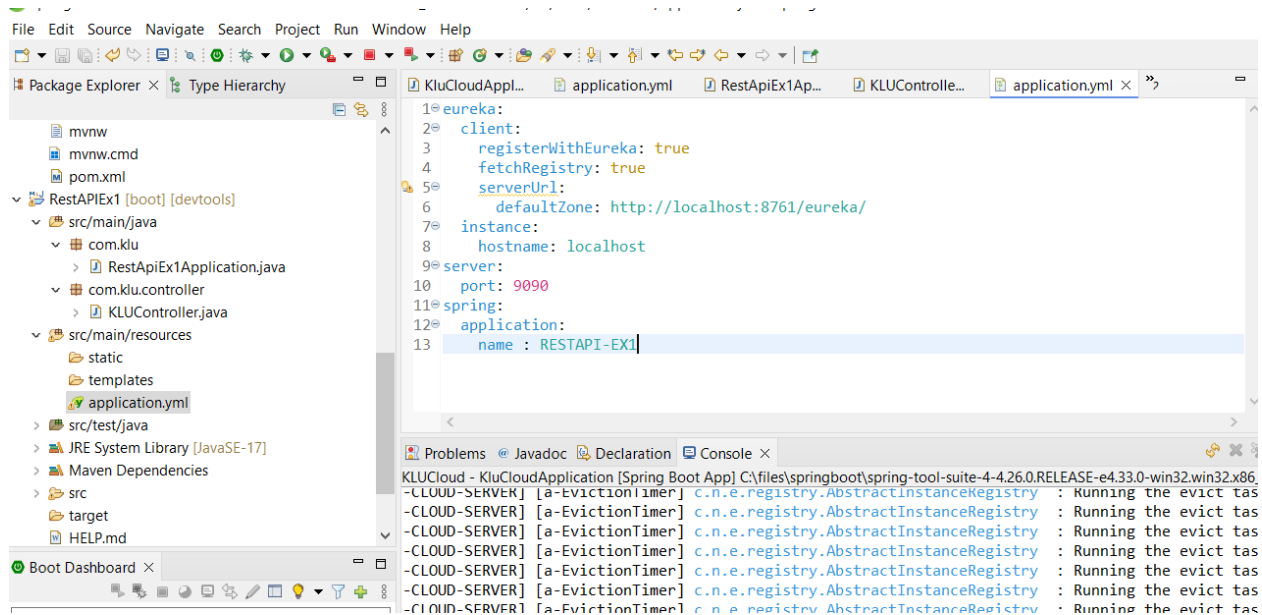
Step-5



Step-6



Step-7



Application.yml
eureka:

```
client:
  registerWithEureka: true
  fetchRegistry: true
  serverUrl:
    defaultZone: http://localhost:8761/eureka/
instance:
  hostname: localhost
server:
  port: 8001
spring:
  application:
    name : RESTAPI-EX2
```

Step-8

← → ↺ localhost:8761 🔍 ☆ ⬇️ ⚠️

🗄️ | 📖 Software Testing Help ⚙️ KL ERP 🌐 Scientific Internatio... 🌐 portal.klef.in/klef 🗣️ Bypass | HIX.AI ⚙️ KLH - Learning Man... 🔔 Notifications | Linke... » | 📁 All B...

[localhost](#)

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
RESTAPI-EX1	n/a (1)	(1)	UP (1) - DESKTOP-MCL27LN-RESTAPI-EX1-9090

General Info

Name	Value
total-avail-memory	88mb
num-of-cpus	4
current-memory-usage	59mb (67%)
server-uptime	00:39
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/,
available-replicas	

Microservices project: Deploy RestAPIEx2 Project in RestAPIEx1

RestAPIEx2:

 — □ × 

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

?

Step-2:

 — □ × 

New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☒ Eureka Discovery Client	☐ Eureka Server	☐ MySQL Driver
☒ Spring Boot DevTools	☐ Spring Data JPA	☒ Spring Web

Available:

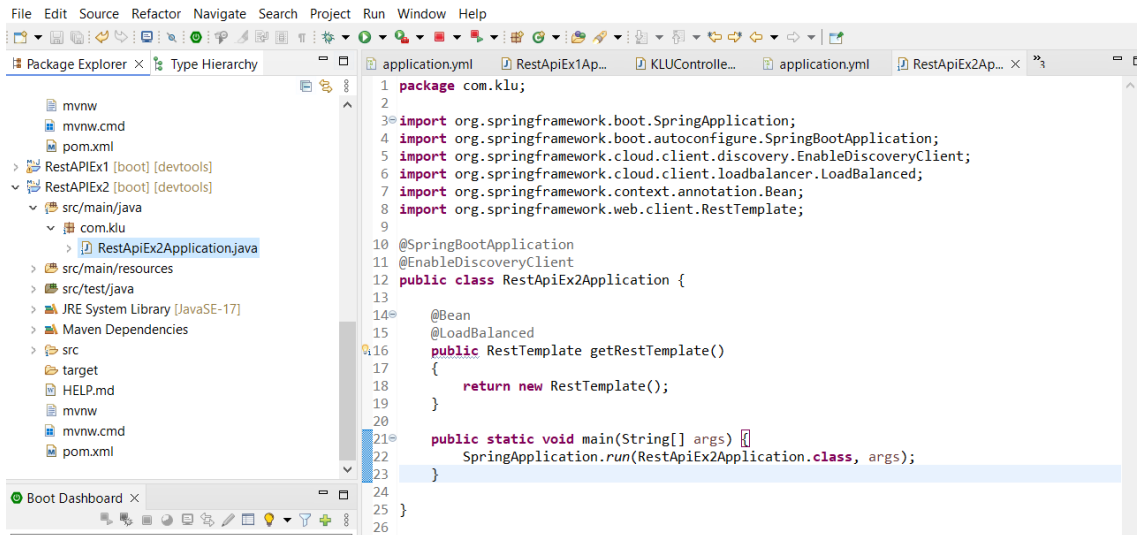
- ▶ AI
- ▶ Developer Tools
- ▶ Google Cloud
- ▶ I/O
- ▶ Messaging
- ▶ Microsoft Azure
- ▶ NoSQL
- ▶ Observability
- ▶ Ops
- ▶ SQL
- ▶ Security

Selected:

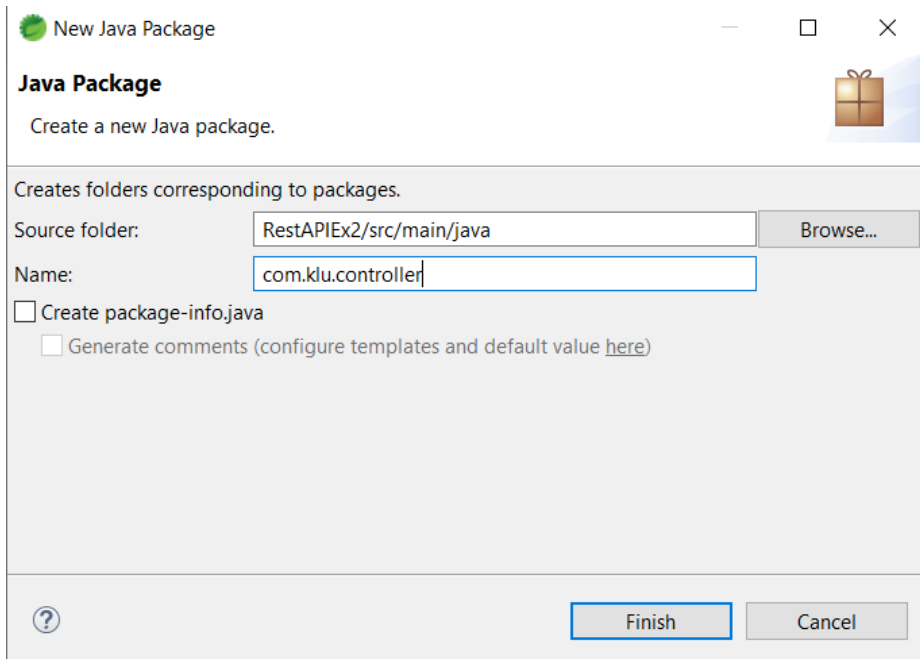
- X Spring Boot DevTools
- X Eureka Discovery Client
- X Spring Web

?

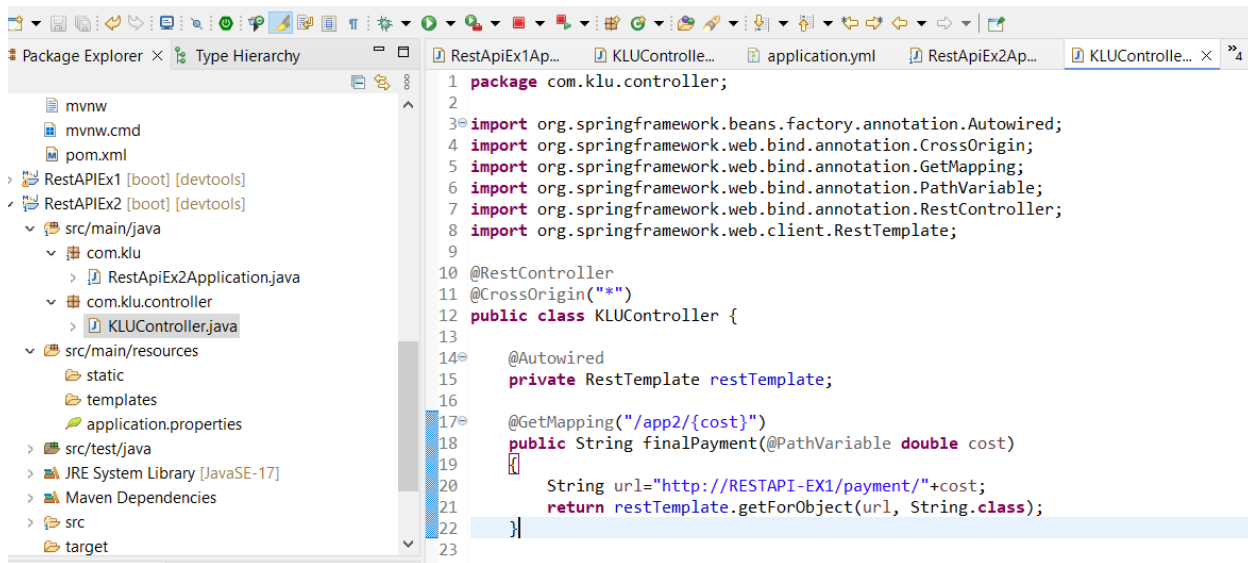
Step-3:



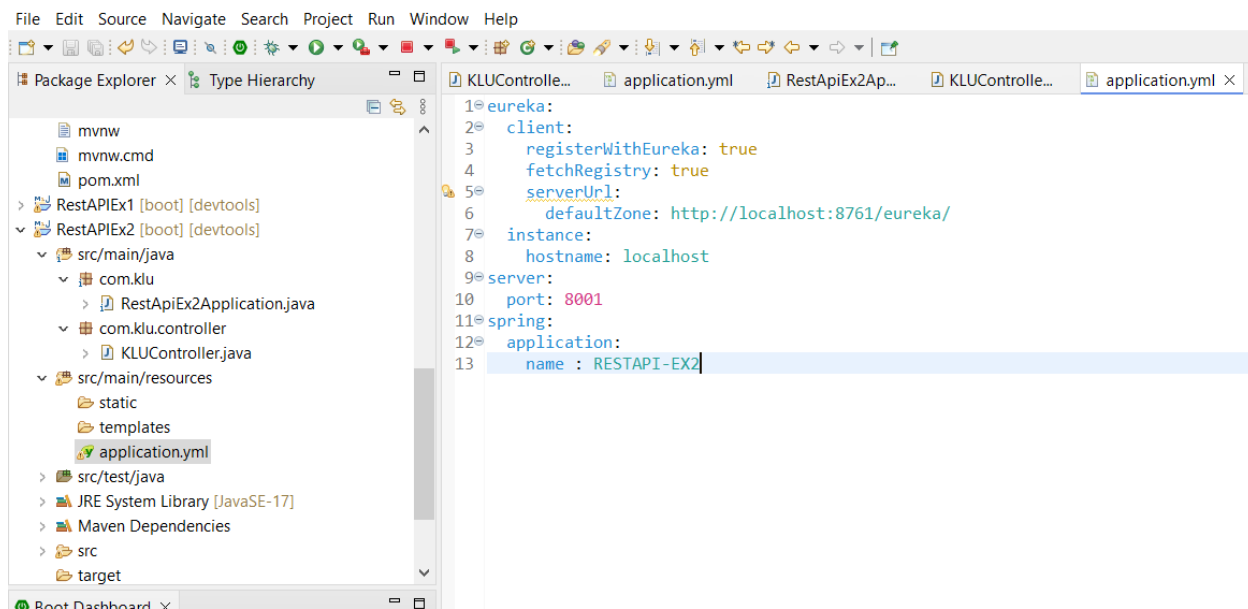
Step-4



Step-5



Step-6



Step-7

localhost:8761

Software Testing HelpKL ERPScientific Internatio...portal.klef.in/klefBypass | HIXAIKLH - Learning Man...Notifications | Linke...All Bookm

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
RESTAPI-EX1	n/a (1)	(1)	UP (1) - DESKTOP-MCL27LN:RESTAPI-EX1:9090
RESTAPI-EX2	n/a (1)	(1)	UP (1) - DESKTOP-MCL27LN:RESTAPI-EX2:8001

Step-8

My WorkspaceNew ImportOverviewGET userregisterNo environment

CollectionsNew CollectionPOST userregister

OverviewGET userregister

ParamsAuthorizationHeaders (8)BodyScriptsSettings

noneform-datax-www-form-urlencodedrawbinaryGraphQLJSON

```
1 {
2   "username": "kluhzayudu",
3   "password": "subbu1234",
4   "email": "subbu1@gmail.com",
5   "role": "fullstack"
6 }
```

BodyCookiesHeaders (8)Test Results200 OK4.24 s273 B

PrettyRawPreviewVisualizeText

1 Final Price ...500.00

